

Arm Cortex-M0 Processor Core and Interrupts

Chapter Contents

Overview	92
CPU Core	92
Concepts	92
Architecture	94
Registers	95
Memory	96
Instructions	100
Thumb Instruction Encoding	106
Operating Behaviors	107
Exceptions and Interrupts	107
CPU Exception Handling Behavior	108
Handler Mode and Stack Pointers	108
Entering a Handler	109
Exiting a Handler	110
Hardware for Interrupts and Exceptions	111
Hardware Overview	111
Exception Sources, Vectors, and Handlers	111
Input and Peripheral Interrupt Configuration	114
NVIC Operation and Configuration	119
Processor Exception Masking	121
Software for Interrupts	122
Program Design	122
Interrupt Configuration	125
Writing ISRs in C	126
Sharing Data Safely Given Preemption	127
Summary	130
Exercises	130
References	131

Overview

In this chapter we examine the processor core, which runs a program by executing its instructions. We learn about the organization of the core, how instructions are executed, how data can be stored and accessed in registers and memory, and what types of operations the processor can perform.

We then examine exceptions and interrupts, which allow a program to respond quickly to events while maintaining a simple software structure. We examine how the processor responds to exceptions and interrupts and then study the hardware support circuits. Finally, we discuss how to write software to configure and use interrupts.

CPU Core

Concepts

Figure 4.1 shows an overview of the components within a microcontroller. These include a processor core (for executing a program), memory (for storing data and instructions), and supporting peripheral components that add functionality or improve performance. In this chapter we study the CPU core, which can be found in the upper left corner of the figure.

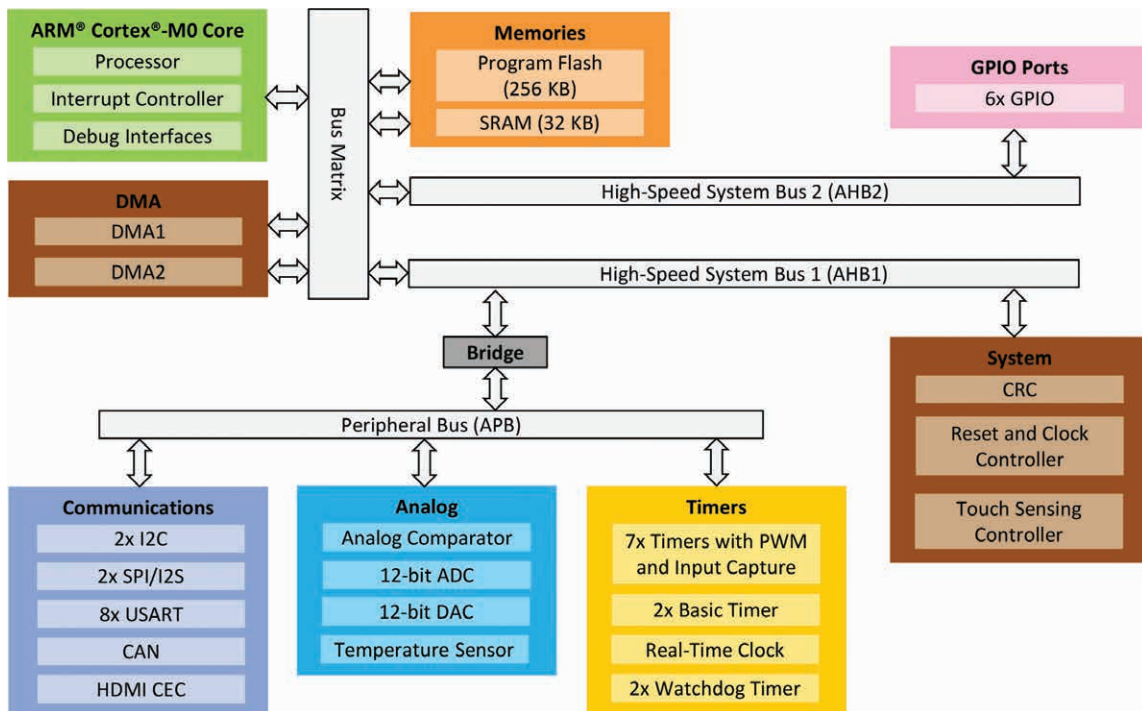


Figure 4.1 Overview of components within an STM32F091RC microcontroller.

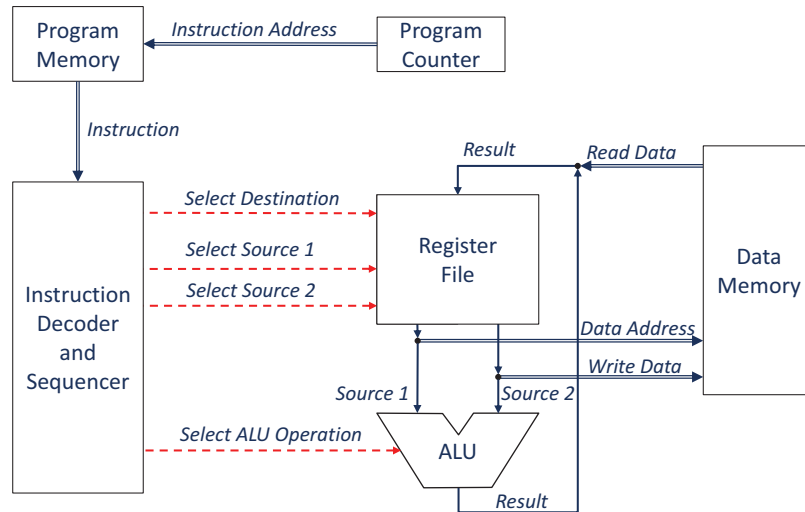


Figure 4.2 Simplified structure of a CPU core.

The processor core executes the **instructions** that make up a program. Figure 4.2 shows the components of a simple CPU core from the instruction-processing point of view. Each instruction specifies an **operation** to perform and which **operands** to use for that operation.

instruction

Command for processor to execute. Consists of an operation and zero or more operands.

operation

Part of an instruction: specifies what work to do

operand

Part of an instruction: parameter used by operation

- The program's instructions are stored in program memory (at the top of the diagram). A **register** called the program counter (PC) specifies the location (address) of the next instruction in that memory to execute.
- The **register file** holds temporary data values before and after the arithmetic/logic unit (ALU) processes them. It is easy to access but is small, holding only a few items.
- The heart of the CPU is the ALU. It performs the actual data processing operations such as addition, subtraction, logic operations (and, or, etc.), comparison, and so on. The ALU gets its operands from the register file (described next) or from within the instruction itself.
- The data memory provides longer-term data storage. It is much larger, holding thousands, millions or more data items. However, it is slower and more complex to access.
- Control logic decodes the instruction into various control and data signal sequences that are sent to other components of the CPU to control their operation.

register*Hardware circuit which can store a data value***register file***Holds CPU's general purpose registers*

Figure 4.2 highlights three different types of information flowing in the CPU as it executes a program:

- Register and ALU data flow, identified with single lines.
- Memory addresses and data, identified with double lines.
- Control and selection signals, identified with dotted lines.

Instruction processing follows this sequence:

- The CPU reads an instruction from the program memory location specified by the program counter.
- The control logic decodes the instruction and uses the resulting information to control other subsystems in the CPU core. This information specifies:
 - Which registers in the register file to read for the instruction's source operands
 - How to generate any other source operands
 - Whether the ALU will perform an operation (and which one), or whether memory will be accessed
 - Which register in the register file will be written with the result
 - How to update the PC

Programs normally follow a sequential instruction execution, by advancing to the instruction located immediately after the current one. However, a control-flow instruction (e.g. branch, subroutine call, return) or interrupt request will make the flow of control jump to a different location, enabling loops, condition tests, subroutine calls, and other behaviors.

Some CPU cores speed up programs by improving this execution sequence. For example, pipelining involves starting to work on the next instruction before the current instruction has completed.

Architecture

The architecture defines several aspects of a processor:

- Programmer's model: operating modes, registers, memory map
- Instruction set architecture (ISA): instructions, addressing modes, data type
- Exception model: interrupt handling
- Debug architecture: debug features.

The Cortex-M0 processors implement the ARMv6-M architecture profile [1], which is a specialized and smaller version of the more general ARMv6 architecture profile. For further information beyond what is presented here, please refer to the existing texts and manuals [2] [3] [4] [5].

In the Arm programming model, only the data located in registers can be processed. Data in memory cannot be processed directly. Instead it must be loaded into registers before processing and perhaps stored back to memory afterwards; hence it is called load/store architecture. This type of architecture simplifies the design of hardware, generally increasing speed and reducing power consumption.

The *native data types* for a CPU core are directly supported by the processor's hardware and execute quickly. The ARMv6-M native data types are signed and unsigned 32-bit values. Operations with other data types can be emulated by multiple software instructions, potentially taking more time.

native data type

Primary data type used by ALU and registers. 32-bit integer for Arm Cortex-M CPUs.

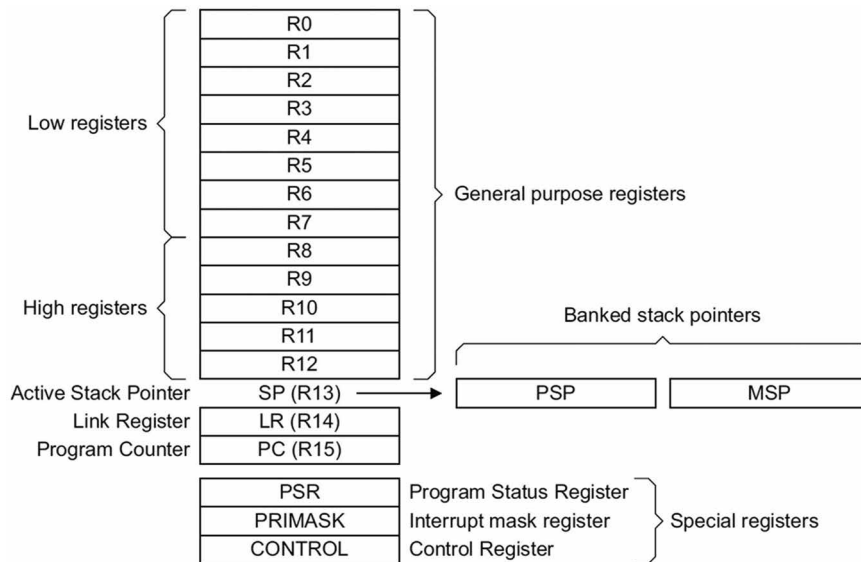


Figure 4.3 Registers in Arm programmer's model. Software tools such as the assembler typically accept both upper and lower case names for the core registers (r0-r15, sp, lr, pc).

Registers

The Arm programmer's model features multiple 32-bit registers, shown in Figure 4.3. Some of them are for general use, whereas others have specific purposes and unique characteristics. Software tools such as the assembler typically recognize both upper and lower case versions of these names (e.g both R0 and r0).

- Registers R0 through R12 are general purpose registers for data processing.
- Register R13 is called the stack pointer (SP), and is used to manage a data storage structure called the stack. SP refers to one of two possible stack pointers, the main stack pointer (MSP)

Code	0x0000 0000 0x1FFF FFFF
SRAM	0x2000 0000 0x3FFF FFFF
Peripheral	0x4000 0000 0x5FFF FFFF
External RAM	0x6000 0000 0x9FFF FFFF
External Device	0xA000 0000 0xDFFF FFFF
Cortex-M0+ Peripherals	0xE000 0000 0xE00F FFFF
System	0xE010 0000 0xFFFF FFFF

Figure 4.5 Memory map for Cortex-M processors.

256 KB Flash (Code region)	0x0800 0000 0x0803 FFFF
32 KB SRAM (SRAM region)	0x2000 0000 0x2000 7FFF

Figure 4.6 Flash ROM and SRAM memories in STM32F091RC MCU.

Endianness

Endianness describes the order in which multi-byte values are stored in memory at a range of addresses. For example, a four-byte value (a word) could be stored at addresses A through A+3. For **little-endian** systems, the **least-significant** byte is stored at the lowest address, as in Figure 4.7. For **big-endian** systems, the **most-significant** byte is stored at the lowest address, as in Figure 4.8

For ARMv6-M systems, instructions are always little-endian. Data can be of either endianness, as determined by the CPU implementation. STM32 microcontrollers built with Cortex-M processors are little-endian for data (and instructions, of course).

endianness

Property which describes order of bytes in multi-byte structures stored in memory

little-endian

Describes byte ordering convention in which least-significant byte is stored first in memory

big-endian

Describes byte ordering convention in which most-significant byte is stored first in memory

least-significant

Having the smallest place value. The least-significant byte of a two-byte value represents values of 0 to 255.

most-significant

Having the greatest place value. The most-significant byte of a two-byte value represents values of 0 to 65,280 which are multiples of 256.

Stack

A **stack** is a data structure that helps programs reuse memory safely. Rather than permanently allocate a memory location for a temporary data value, a stack allows the program to allocate a location, use it, and then free it when done, allowing for later reuse of that location for other purposes.

Adding data to a stack is called **pushing** data onto the stack, whereas removing data is called **popping** data.

stack

Last-in, first-out data structure. Data items are removed (popped) in the opposite order they were inserted (pushed).

push

Instruction which writes a data item next free stack location in memory and updates the stack pointer

pop

Instruction which reads a data item from the top of the stack (last used location) in memory and updates the stack pointer

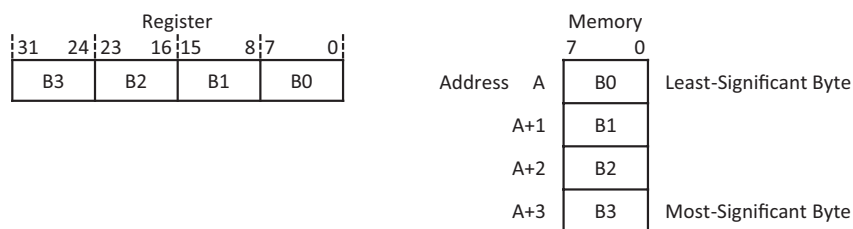


Figure 4.7 With little-endian storage, the lowest memory address holds the least-significant byte.

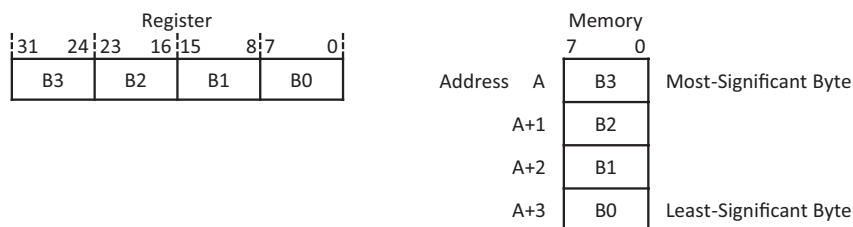


Figure 4.8 With big-endian storage, the lowest memory address holds the most-significant byte.

Stacks use a “last-in, first-out” data organization. If item X and then item Y are pushed onto the stack, then the first pop will return Y, and the second pop will return X.

The SP (R13) points to the last item on the stack, not the first free location. Pushing data decreases the SP value by the number of bytes of data added. Popping data increases SP value by the number of bytes removed. This means that the stack grows toward smaller addresses.

Let’s walk through an example of stack use. We begin with the item D on the top of the stack at address A, as shown in Figure 4.9. We push a data item X (Figure 4.10), and then push another data item Y (Figure 4.11). The first time we pop an item off the stack, we get Y (Figure 4.12) and the next pop results give the value X (Figure 4.13). The push or pop operation adjusts the stack pointer SP to point to the top item of the stack.

Memory Address	Contents
0x2000 0000	Free space
0x2000 0004	Free space
0x2000 0008	Free space
0x2000 000c	Free space
0x2000 0010	D (existing data)

SP before pushing data onto stack →

Figure 4.9 Initially stack has one data item (D). SP points to D, the top of stack.

Memory Address	Contents
0x2000 0000	Free space
0x2000 0004	Free space
0x2000 0008	Free space
0x2000 000c	X
0x2000 0010	D (existing data)

SP after pushing data onto stack →

Figure 4.10 Stack after pushing item X. SP now points to X, the new top of stack.

Memory Address	Contents
0x2000 0000	Free space
0x2000 0004	Free space
0x2000 0008	Y
0x2000 000c	X
0x2000 0010	D (existing data)

SP after pushing data onto stack →

Figure 4.11 Stack after pushing item Y. SP now points to Y.

Memory Address	Contents
0x2000 0000	Free space
0x2000 0004	Free space
0x2000 0008	Free space
0x2000 000c	X
0x2000 0010	D (Existing data)

SP after popping data from stack →

Figure 4.12 Stack after popping one item from stack. Popped value is Y. SP now points to X.

Memory Address	Contents
0x2000 0000	Free space
0x2000 0004	Free space
0x2000 0008	Free space
0x2000 000c	Free space
SP after popping data from stack → 0x2000 0010	D (Existing data)

Figure 4.13 Stack after popping another item from stack. Popped value is X. SP points to D again.

For the Arm Cortex-M, all pushes and pops use 32-bit data items; no other size is possible. Since all possible stack pointer values are multiples of four, the hardware is designed so that the two least significant bits of the stack pointer are always zeros.

Instructions

9800	LDR	R0, [SP, #0]
1900	ADDS	R0, R0, R4
9000	STR	R0, [SP, #0]

Listing 4.1 Instructions in machine language (first column) and assembly language (remaining columns).

An instruction must specify which operation to perform, and possibly data and parameters for the operation. The instruction may be written in *machine language* or *assembly language*.

machine language

Code in which each instruction is represented as a numerical value. Processed directly by CPU.

assembly language

Human-readable representation of machine code

The machine language form is a number that is stored in program memory and which the CPU can decode quickly and easily. For example, in Listing 4.1, we see three instructions in machine language: 9800, 1900, and 9000.

Assembly language is a text form that is more easily understood than machine language, which is tedious to read, write, and edit. An assembly language instruction uses mnemonics to specify the operation and any operands. The first operand usually specifies the destination of the operation, which will be overwritten with the result. The following operands usually specify the sources of the input values. For some instructions, the destination or source is implied by the instruction itself. A software tool called an *assembler* translates the assembly language code into machine language code for the CPU to execute.

assembler

Software tool which translates assembly language code into machine code

Listing 4.1 also shows the assembly language forms of the same three instructions:

- **LDR R0, [SP, #0]** loads register R0 from memory starting at address **SP+0**.
- **ADDS R0, R0, R4** adds the contents of R0 and R4, placing the results in R0.
- **STR R0, [SP, #0]** saves the contents of R0 to memory starting at address **SP+0**.

Operands may be located in a general-purpose register (R0 through R12), in the instruction word itself, in a condition code flag or in memory. Most instructions can access operands only in registers, the instruction word, or condition code flags. Some instructions (load, store, push, and pop) are able to access operands in memory.

Table 4.1 provides an overview of the different instructions available for the Cortex-M0. Some of the instructions have two versions, one that updates the condition code flags in the APSR (indicated with an S suffix after the instruction mnemonic) and one that does not (no S suffix). Full details of the instruction set are presented in the documentation [2] [4].

Table 4.1 Summary of Cortex-M0 Instructions

Category	Instruction Type	Instruction Mnemonic
Data Movement	Move	MOV, MOVS, MRS, MSR
Data Processing	Math	ADD, ADDS, ADCS, ADR, MULS, RSBS, SBCS, SUB, SUBS
	Logic	ANDS, EORS, ORRS, BICS, MVNS, TST
	Compare	CMP, CMN
	Shift and Rotate	ASRS, LSLS, LSRS, RORS
	Extend	SXTB, SXTH, UXTB, UXTH
	Reverse	REV, REV16, REVSH
Memory Access	Load	LDR, LDRB, LDRH, LDRSH, LDRSB, LDM
	Store	STR, STRB, STRH, STM
	Stack	PUSH, POP
Control Flow	Branch	B, BL, BX, BLX, BEQ, BNE, BCS, BHS, BCC, BLO, BMI, BPL, BVS, BVC, BHI, BLS, BGE, BLT, BGT, BLE
Miscellaneous		BKPT, CPSID, CPSIE, WFE, WFI, SVC, YIELD, DMB, DSB, ISB, SEV, NOP

Data Movement Instructions

The **MOV** and **MOVS** instructions move data to a general-purpose register and have several forms:

- **MOV R3, R5** copies the data from register R5 into register R3.
- **MOV R2, #151** copies the value **151** into register R2. The value to be moved is called an **immediate value** because it is located in the instruction itself, in an 8-bit field. Unsigned values from 0 to 255 can be loaded into registers in this way. Larger values must be loaded from memory using the **LDR** instruction, described later.

immediate value

Data value which is stored as part of a machine instruction

The special registers (such as CONTROL, PRIMASK, xPSR) are accessed using **MSR** and **MRS** instructions. **MRS** moves the data from a special register into a general-purpose register, whereas **MSR** does the opposite.

Data-processing Instructions

Data-processing instructions include both math and logic operations. These are performed on one or more 32-bit data values, typically two registers or one register and an immediate constant.

Math operations include addition (with and without carry), subtraction (with and without borrow), subtraction with reversed operands, and multiplication:

- **ADDS R0, R1, R2** adds the contents of R1 and R2, writing the result to R0 and updating the condition code flags in the APSR.

Logic operations include bitwise AND, bitwise AND with complement, exclusive OR complement, and test.

- **ANDS R4, R3, R7** computes the bitwise and of registers R3 and R7, writing the result to R4 and updating the condition code flags in the APSR.

Compare instructions calculate the difference between two operands, set the condition code flags accordingly, but discard the calculated difference. A test instruction performs a logical AND on two operands and then sets the condition code flags. Condition code flags are explained in depth in the documentation.

- **CMP R3, R5** computes R3–R5 and sets the flags according to the answer. For example, if R3 holds 61 and R5 holds 29, then the comparison results in a value of $61 - 29 = 32$. This will clear all flags: N, Z, C, and V, as the result is not negative, not zero, no carry occurred, and no overflow occurred.

Shift and **rotate** instructions shift a word left or right by the specified number of bits. Both logical and arithmetic operations are supported.

- **LSRS R1, #1** shifts register R1 to the right by one bit position. Because this is a logical shift, the MSB is loaded with zero. Bit 0 is shifted into the carry flag.

Extend instructions convert an 8- or 16-bit value to fill a 32-bit register. The upper bits can be filled in one of two ways, depending on whether the original value is signed or unsigned. An **unsigned** extend instruction (**UXTB** or **UXTH**) will fill the upper bits with zeros, preserving the value of unsigned data. A **signed** extend instruction (**SXTB** or **SXTH**) will fill the upper bits with the most significant bit of the original value, preserving the value of the signed data.

unsigned

Numbering system which is able to represent positive values and zero

signed

Numbering system which is able to represent positive and negative values and zero

Reverse instructions can reverse the byte order in a 32-bit word or two 16-bit half-words, providing conversion between little-endian and big-endian data.

Memory-Access Instructions

Recall that in the Arm programming model, data in memory must be loaded into registers before it can be processed, and then may need to be stored back into memory.

Memory-addressing

Memory accesses use offset addressing, in which a base register R_n and an offset are added together to create the actual address for the memory access. The offset can be another register or an immediate constant value (which is stored in the instruction word). The base register is not modified by the address calculation.

The assembly code syntax for addressing memory is a bracketed expression. For example:

- **[R0]** indicates the memory location starting at the address in register R0. If R0 has a value of 4000, then this indicates the memory value starting at address 4000.
- **[R0, #22]** indicates the memory location starting at the address which is the sum of the value in R0 and 22. If R0 has a value of 4000, then this indicates the memory value starting at address 4022.
- **[R0, R3]** indicates the memory location starting at the address which is the sum of the values in registers R0 and R3. If R0 has a value of 4000 and R3 has a value of -80, then this indicates the memory value starting at address 3920.

Load/Store Instructions

The **LDR** instruction loads a register with a 32-bit word from the four memory locations starting at the source address. The **STR** instruction stores the 32-bit contents of a register to the four memory locations starting at the destination address.

- **LDR R0, [R4, #8]** will load register R0 with the contents of the memory word (4 bytes) starting at location $R4 + 8$.
- **STR R1, [R4, R5]** will store register R1 to the memory word (4 bytes) starting at location $R4 + R5$.

Data smaller than 32 bits can be loaded or stored from memory as well. Byte (8 bits) and half-word (16 bits) loads and stores are supported. The **STRB** operation stores the least-significant byte of a register at the destination memory address, whereas **STRH** stores the least-significant half-word in the two bytes starting at the destination address.

Loads are more complex because an 8- or 16-bit value from memory does not completely fill a 32-bit register. This is similar to the signed and unsigned extension instructions described earlier. A load register unsigned instruction (**LDRB** or **LDRH**) will fill the upper bits with zeros, preserving the value of the unsigned data. A load register signed instruction (**LDRSB** or **LDRSH**) will fill the upper bits with the most significant bit of the value loaded from memory, preserving the value of signed data.

The ARMv6-M architecture does not support unaligned data accesses to memory. When accessing a word (which is four bytes) in memory, the starting address must be a multiple of four. When accessing a half-word (two bytes), the starting address must be a multiple of two.

The instruction set also offers load/store multiple instructions to reduce the amount of code and time needed by the program. The **LDM** operation will load the registers specified in an operand list from memory starting at the given address. **STM** will store multiple registers to memory starting at the given address.

Stack Instructions

The Arm ISA supports a stack in memory to simplify subroutine calls and returns. The SP register points to the last data item on the stack. Pushing another item will write to a lower address in memory. This is called a “full-descending” stack.

The **PUSH** operation pushes (writes) selected registers onto the stack and updates the stack pointer. Source registers R0 through R7 and the link register (LR) can be specified in the instruction. With each register pushed to memory, the CPU decrements the stack pointer by four bytes. Regardless of the operand ordering in the instruction, the registers are always pushed in the same order: the largest register number is pushed first and goes to the largest address in memory.

- **PUSH {R0, R5–R7}** will push registers R0, R5, R6, and R7 onto the stack, and subtract 16 from SP.

The **POP** operation pops (reads) selected registers from the stack and updates the stack pointer. Destination registers R0 through R7 and the PC can be specified in the instruction. With each register popped from memory, the CPU increments the stack pointer by four bytes. Regardless of the operand ordering in the instruction, the registers are always popped in the same order: the smallest register number is popped first and comes from the smallest address in memory.

- **POP {R2, R4}** will load registers R2 and R4 from the stack and add 8 to SP.

Popping a value into the PC will change the program’s flow of control. This can be used to make the CPU return from a subroutine to the calling routine, which will be discussed later.

Control Flow Instructions

Control flow instructions allow programs to repeat code in a loop or execute a selected section of the code based on a conditional test. This is done by changing the PC to a different value.

An unconditional branch always transfers program execution to the specified destination address.

- **B Target_label** will cause the program to start executing code at (branch to) the program location called **Target_label**.

A conditional branch will transfer program execution to the specified destination address if a given condition is true. The operation consists of a **B** followed by a condition code suffix (e.g. **BEQ**, **BNE**). This suffix specifies which particular combination of the condition code flags (N, Z, C, V) to evaluate. The conditional branch instruction executes if the condition code flags match the specified condition. Figure 4.14 shows the meanings of the different condition code suffixes.

The condition suffixes correspond to the flag settings after a compare instruction has been executed. Earlier we presented the instruction **CMP R3, R5**. If R3 holds 61 and R5 holds 29,

then the comparison results in a value of $61 - 29 = 32$. This will clear the flags N, Z, V, and C. Let's see how different types of conditional branch would behave:

- Branch if equal: **BEQ Target_label** will not branch to the target, since Z is not one. This is correct; 61 is not equal to 29.
- Branch if not equal: **BNE Target_label** will branch to the target, since Z is zero. This is correct; 61 is not equal to 29.
- Branch if greater than: **BGT Target_label** will branch to the target, since Z is zero and both N and V have the same value (zero in this case). This is correct; 61 is greater than 29.
- Branch if greater than or equal: **BGE Target_label** will branch to the target, since both N and V have the same value (zero in this case). This is correct; 61 is greater than or equal to 29.
- Branch if less than: **BLT Target_label** will not branch to the target, since N and V do not have different values. This is correct; 61 is not less than 29.

Note that the conditional branch can be performed after any instruction, not just a compare. It will evaluate the current values of the condition code flags. Remember that some instructions do not update the condition code flags.

Another useful control flow concept is the subroutine. Multiple functions may need to perform similar work. Rather than duplicate the instructions for that work in each function, the instructions may be placed into a subroutine that may be called by different functions as needed. Only

Suffix	Flags	Meaning
EQ	Z = 1	Equal, last flag setting result was zero.
NE	Z = 0	Not equal, last flag setting result was non-zero.
CS or HS	C = 1	Higher or same, unsigned.
CC or LO	C = 0	Lower, unsigned.
MI	N = 1	Negative.
PL	N = 0	Positive or zero.
VS	V = 1	Overflow.
VC	V = 0	No overflow.
HI	C = 1 and Z = 0	Higher, unsigned.
LS	C = 0 or Z = 1	Lower or same, unsigned.
GE	N = V	Greater than or equal, signed.
LT	N != V	Less than, signed.
GT	Z = 0 and N = V	Greater than, signed.
LE	Z = 1 or N != V	Less than or equal, signed.
AL	Can have any value	Always. This is the default when no suffix is specified.

Figure 4.14 Condition code suffixes indicate which flags to test [3].

one version of the instructions needs to be stored (reducing code size) and maintained (reducing development time). Software developers use subroutines to create building blocks, enabling them to think at a higher level when developing a system.

When a function calls the subroutine, the subroutine executes, and then the calling function resumes execution at the instruction following the subroutine call. Subroutine calls are performed using a branch and link (BL) or a branch and link with exchange (BLX). These are similar to the unconditional branch (B) with one major difference. When a function calls a subroutine, it expects to resume execution after the subroutine completes. The *return address* indicates the location of the next instruction in that function to execute after the subroutine completes. The return address is stored in the LR when a BL or BLX instruction is executed.

return address

Address of next instruction to execute after completing a subroutine

- **BL Subroutine1** will call **Subroutine1** and save the return address in the link register.
- **BLX R0** will call the code with the address specified by R0 and save the return address in the link register. R0 needs to have been loaded already with the address of the subroutine.

Returning from the subroutine requires copying the return address to the program counter. When LR holds the return address, this can be done with **BX LR**. If this subroutine (**Sub_1**) may call another subroutine (**Sub_2**), then **Sub_1** will save the LR onto the stack before calling **Sub_2**. **Sub_1** returns by popping the saved LR value from the stack into the PC using the **POP {PC}** instruction.

Miscellaneous Instructions

The **CPSID** and **CPSIE** instructions are used to control the PRIMASK register, which determines whether the CPU responds to interrupts and certain other exceptions. **CPSIE** enables the response, whereas **CPSID** disables the response.

NOP is an instruction that does nothing (no operation). It can be used to align instructions in memory or delay program execution.

There are additional instructions that support debugging, exceptions, sleep modes, complex memory systems, and signaling, but we do not discuss them here.

Thumb Instruction Encoding

The full Arm instruction set uses 32 bits to represent each instruction, but this makes programs larger and raises costs, which are often crucial for embedded systems. The ARMv6-M profile only supports the Thumb instruction set, a subset of the full Arm instruction set in which most instructions are represented as 16-bit half-words. This reduces program memory requirements significantly and usually allows instructions to be fetched faster. The main limitation of the 16-bit instruction is that there are fewer bits available to represent the operation and operands. As a result, some 32-bit Arm instructions and advanced features are not available. For example, most 16-bit Thumb instructions can access only registers R0 through R7, but not R8 through R13. As a result, a program using 16-bit Thumb instructions may require more instructions

(and likely execution cycles), but it will still be much smaller than a program with 32-bit Arm instructions.

Operating Behaviors

A CPU may have several operating modes with different capabilities to provide better performance, more safety, or additional features.

Thread and Handler Modes

The processor normally runs in Thread mode, but enters Handler mode when servicing exceptions and interrupts. The Thread mode simplifies the creation of multitasking systems. Thread and Handler modes differ in stack pointer use, which is described in the section “Handler Mode and Stack Pointers” below.

Instruction Execution versus Debugging

Normally the CPU runs non-stop, executing a program instruction by instruction unless it is in sleep mode or halted. However, there is a debug state in which the CPU does not execute instructions, but is instead controlled by a debugger circuit. Because this debugger hardware has full access to registers and memory, the target program does not need to be modified, simplifying the development tools.

Exceptions and Interrupts

Exceptions and interrupts are critical tools for making an embedded system responsive while supporting concurrent operation of hardware and software. In the Arm programmer’s model, interrupts are a type of exception.

Events such as hardware signals or anomalous program conditions can trigger exceptions. A peripheral or external device sends a hardware signal to the exception controller hardware to indicate that an event has occurred. The processor then services (handles) the exception with these steps:

- Pauses the program.
- Saves context information, such as registers and which instruction to execute next in the current program.
- Determines which handler (also called a service routine) to run. Each type of exception or interrupt can have a separate handler.
- Runs the code for the handler.
- Restores the context information that was saved previously. The processor then resumes executing the current program where it left off.

The CPU performs most of this work automatically in hardware, making the system more responsive. The only work performed in software is running the handler.

Exceptions provide efficient event-based processing, unlike polling that can waste many CPU cycles. Exceptions provide a quick response to events regardless of program complexity, location,